EXPT NO: 2	**TRANSPORTATION PROBLEM**
DATE:	

AIM:

To solve the Transportation Problem for minimizing the cost of transporting goods from multiple sources to multiple destinations using Python.

PROCEDURE:

1. **Install Required Libraries:**

o Make sure you have numpy and scipy installed. You can install them using pip if not already installed.

bash

Copy code

pip install numpy scipy

2. **Define the Problem:**

o Identify the cost matrix, supply array, and demand array.

3. **Set Up the Problem in Python:**

o Use the scipy.optimize.linprog function to set up and solve the Transportation Problem.

4. **Run the Code:**

o Execute the Python code to get the optimized result.

5. **Analyze the Output:**

o Interpret the results and validate the solution against the problem constraints.

CODE

PROGRAM 1:

```
import numpy as np
from scipy.optimize import linprog

def transportation_problem(supply, demand, costs):
# Balance supply and demand
if sum(supply) != sum(demand):
        if sum(supply) < sum(demand):
        diff = sum(demand) - sum(supply)
        supply.append(diff)
        costs = np.vstack((costs, np.zeros((1, costs.shape[1]))))
else:
        diff = sum(supply) - sum(demand)
        demand.append(diff)
        costs = np.hstack((costs, np.zeros((costs.shape[0], 1))))

num_sources = len(supply)
num_destinations = len(demand)
 # Create equality constraint matrix
A_eq = np.zeros((num_sources + num_destinations, num_sources * num_destinations))
 # Supply constraints
for i in range(num_sources):
A_eq[i, i * num_destinations:(i + 1) * num_destinations] = 1
# Demand constraints
for j in range(num_destinations):
```

```
for i in range(num_sources):
        A_eq[num_sources + j, i * num_destinations + j] = 1
# Right-hand side
b_eq = np.concatenate((supply, demand))
# Objective function
c = costs.flatten()
# Solve LP
result = linprog(c, A_eq=A_eq, b_eq=b_eq, method='highs')
# Extract results
allocation = result.x.reshape(num_sources, num_destinations)
total_cost = result.fun
return allocation, total_cost


 # Example input
supply = [30, 20, 50]
demand = [40, 70, 30]
costs = np.array([
[6, 8, 10],
[9, 12, 13],
[14, 16, 18]
])

# Solve
allocation, total_cost = transportation_problem(supply, demand, costs)
```

```
# Output
print("Optimal Allocation:")
print(allocation)
print("Total Cost:", total_cost)
```

OUTPUT:

```
Optimal Allocation:
[[30.  0.  0.]
[10. 10.  0.]
[ 0. 60. 30.]]
Total Cost: 1900.0
```

PROBLEM 2:

```
import numpy as np
from scipy.optimize import linprog

def transportation_problem(supply, demand, costs):
    # Balance supply and demand
    if sum(supply) != sum(demand):
        if sum(supply) < sum(demand):
            diff = sum(demand) - sum(supply)
            supply.append(diff)
            costs = np.vstack((costs, np.zeros((1, costs.shape[1]))))
        else:
            diff = sum(supply) - sum(demand)
            demand.append(diff)
```

```
        costs = np.hstack((costs, np.zeros((costs.shape[0], 1))))


    num_sources = len(supply)

    num_destinations = len(demand)


    # Create equality constraint matrix

    A_eq = np.zeros((num_sources + num_destinations, num_sources *
num_destinations))

    # Supply constraints

    for i in range(num_sources):

        A_eq[i, i * num_destinations:(i + 1) * num_destinations] = 1

    # Demand constraints

    for j in range(num_destinations):

        for i in range(num_sources):

            A_eq[num_sources + j, i * num_destinations + j] = 1

    # RHS

    b_eq = np.concatenate((supply, demand))

    # Objective function

    c = costs.flatten()

    # Solve

    result = linprog(c, A_eq=A_eq, b_eq=b_eq, method='highs')

    # Results

    allocation = result.x.reshape(num_sources, num_destinations)

    total_cost = result.fun

    return allocation, total_cost
```

```
# Example input
supply = [25, 35, 40]
demand = [30, 40, 30]
costs = np.array([
    [8, 6, 10],
    [9, 12, 13],
    [14, 9, 16]
])
# Solve
allocation, total_cost = transportation_problem(supply, demand, costs)
# Output
print("Optimal Allocation:")
print(allocation)
print("Total Cost:", total_cost)
```

OUTPUT:

```
Optimal Allocation:
[[ 0. 25.  0.]
 [30.  5.  0.]
 [ 0. 10. 30.]]
Total Cost: 945.0
```

RESULT:

Thus The Transportation Problem For Minimizing The Cost Of Transporting Goods From Multiple Sources To Multiple Destinations Using Python Was Executed Successfully.